Rapport – Atelier 1 SQL Complet

Requêtes, calculs, optimisation et sécurité · Thème : gestion commerciale (clients, commandes, produits, paiements)

1. Objectifs

- Concevoir une base de données relationnelle cohérente
- Manipuler les jointures, les fonctions SQL, les opérateurs et les agrégations
- Utiliser `IF()`, `LIMIT`, `DATEDIFF()` et les fonctions de date
- Améliorer la performance avec des index et des vues
- Gérer les droits d'accès (DCL)

2. Environnement

- **Serveur** : MySQL 8.4.11 hébergé sur **Aiven** (cloud), connexion chiffrée SSL (`--ssl-mode=REQUIRED`)
- **Client** : MySQL en ligne de commande
- **Base de travail** : `atelier_commerce`

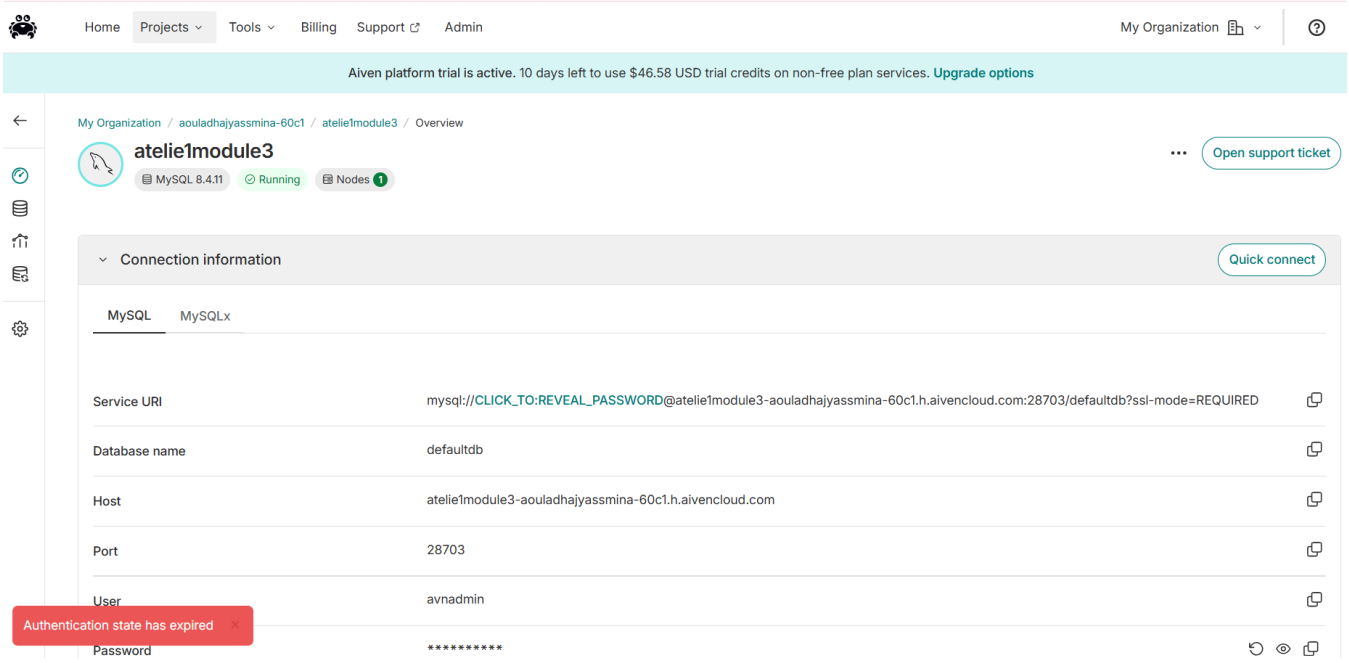


Figure 1 – Tableau de bord Aiven : service MySQL 8.4.11 en état Running (mot de passe masqué).

Connexion au serveur distant :

```
mysql -h <host>.aivencloud.com -P <port> -u avnadmin -p --ssl-mode=REQUIRED
defaultdb
```

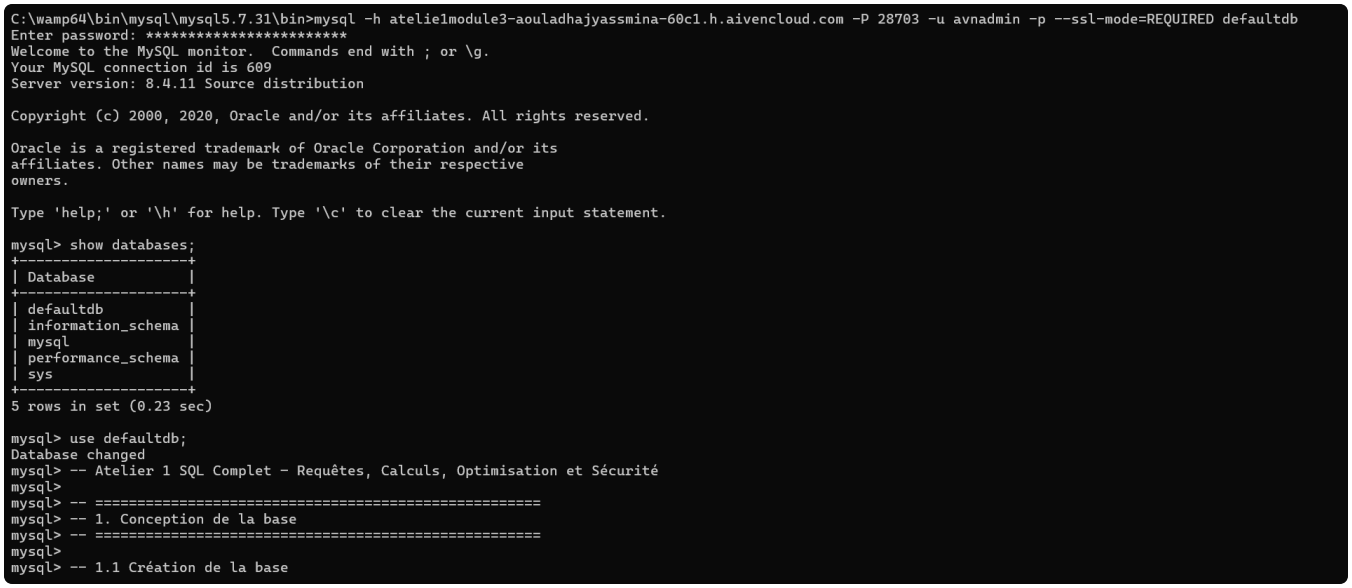


Figure 2 – Connexion réussie au serveur MySQL 8.4.11 d'Aiven depuis mon PC, puis `SHOW DATABASES`.

3. Conception de la base

Modèle relationnel

- `clients` (id, nom, ville)
- `produits` (id, nom_produit, prix_unitaire)
- `commandes` (id, client_id → clients, date_commande)
- `details_commandes` (id, commande_id → commandes, produit_id → produits, quantite)
- `paiements` (id, commande_id → commandes, montant, mode_paiement, date_paiement)

Relations : un client passe plusieurs commandes ; une commande contient plusieurs produits grâce à la table d'association `details_commandes` ; une commande peut avoir un paiement.

```
CREATE DATABASE atelier_commerce;
USE atelier_commerce;

CREATE TABLE clients (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(50),
  ville VARCHAR(50)
);

CREATE TABLE produits (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom_produit VARCHAR(50),
  prix_unitaire DECIMAL(10,2)
);

CREATE TABLE commandes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  client_id INT,
  date_commande DATE,
  FOREIGN KEY (client_id) REFERENCES clients(id)
);

CREATE TABLE details_commandes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  commande_id INT,
  produit_id INT,
  quantite INT,
  FOREIGN KEY (commande_id) REFERENCES commandes(id),
  FOREIGN KEY (produit_id) REFERENCES produits(id)
);

CREATE TABLE paiements (
  id INT AUTO_INCREMENT PRIMARY KEY,
  commande_id INT,
  montant DECIMAL(10,2),
  mode_paiement VARCHAR(30),
  date_paiement DATE,
  FOREIGN KEY (commande_id) REFERENCES commandes(id)
);
```

```
mysql> CREATE DATABASE atelier_commerce;
Query OK, 1 row affected (0.18 sec)

mysql> USE atelier_commerce;
Database changed
mysql>
mysql> -- 1.2 Création des tables
mysql> CREATE TABLE clients (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   nom VARCHAR(50),
    ->   ville VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.51 sec)

mysql>
mysql> CREATE TABLE produits (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   nom_produit VARCHAR(50),
    ->   prix_unitaire DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.09 sec)

mysql>
mysql> CREATE TABLE commandes (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   client_id INT,
    ->   date_commande DATE,
    ->   FOREIGN KEY (client_id) REFERENCES clients(id)
    -> );
Query OK, 0 rows affected (0.10 sec)

mysql>
mysql> CREATE TABLE details_commandes (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   commande_id INT,
    ->   produit_id INT,
    ->   quantite INT,
    ->   FOREIGN KEY (commande_id) REFERENCES commandes(id),
    ->   FOREIGN KEY (produit_id) REFERENCES produits(id)
```

Figure 3 – Création de la base `atelier_commerce` et des tables.

4. Insertion des données

Jeu de données : 4 clients (Alice, Bob, Charlie, Dina), 4 produits (Ordinateur 950, Souris 20, Clavier 45, Écran 250), 4 commandes, 5 lignes de détail et 3 paiements.

```
INSERT INTO clients (nom, ville)
VALUES ('Alice','Paris'), ('Bob','Lyon'), ('Charlie','Toulouse'),
('Dina','Nice');

INSERT INTO paiements (commande_id, montant, mode_paiement, date_paiement)
VALUES (1,990,'Carte','2025-01-12'), (2,45,'Espèces','2025-01-21'),
(3,500,'Virement','2025-02-05');
```

```
    -> );
Query OK, 0 rows affected (0.10 sec)

mysql>
mysql> -- 1.3 Insertion de données
mysql> INSERT INTO clients (nom, ville)
    -> VALUES ('Alice', 'Paris'), ('Bob', 'Lyon'), ('Charlie', 'Toulouse'), ('Dina', 'Nice');
Query OK, 4 rows affected (0.10 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO produits (nom_produit, prix_unitaire)
    -> VALUES ('Ordinateur', 950), ('Souris', 20), ('Clavier', 45), ('Écran', 250);
Query OK, 4 rows affected (0.10 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO commandes (client_id, date_commande)
    -> VALUES (1, '2025-01-10'), (1, '2025-01-20'), (2, '2025-02-01'), (3, '2025-02-10');
Query OK, 4 rows affected (0.10 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO details_commandes (commande_id, produit_id, quantite)
    -> VALUES (1,1,1), (1,2,2), (2,3,1), (3,4,2), (4,2,3);
Query OK, 5 rows affected (0.10 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO paiements (commande_id, montant, mode_paiement, date_paiement)
    -> VALUES (1,990,'Carte','2025-01-12'),
    ->        (2,45,'Espèces','2025-01-21'),
    ->        (3,500,'Virement','2025-02-05');
Query OK, 3 rows affected (0.10 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- ======================================================
mysql> -- 2. Requêtes de jointures
mysql> -- ======================================================
mysql>
```

Figure 4 – Insertion des données dans les cinq tables.

5. Requêtes de jointures

INNER JOIN : produits commandés par client

```
SELECT c.nom, cmd.id AS commande, p.nom_produit, dc.quantite
FROM clients c
INNER JOIN commandes cmd ON c.id = cmd.client_id
INNER JOIN details_commandes dc ON cmd.id = dc.commande_id
INNER JOIN produits p ON dc.produit_id = p.id;
```

nom	commande	nom_produit	quantite
Alice	1	Ordinateur	1

nom	commande	nom_produit	quantite
Alice	1	Souris	2
Alice	2	Clavier	1
Bob	3	Écran	2
Charlie	4	Souris	3

Seuls les clients ayant des commandes apparaissent : Dina est absente.

```
mysql>
mysql> -- INNER JOIN
mysql> SELECT c.nom, cmd.id AS commande, p.nom_produit, dc.quantite
    -> FROM clients c
    -> INNER JOIN commandes cmd ON c.id = cmd.client_id
    -> INNER JOIN details_commandes dc ON cmd.id = dc.commande_id
    -> INNER JOIN produits p ON dc.produit_id = p.id;
+---------+----------+-------------+----------+
| nom     | commande | nom_produit | quantite |
+---------+----------+-------------+----------+
| Alice   |        1 | Ordinateur  |        1 |
| Alice   |        1 | Souris      |        2 |
| Alice   |        2 | Clavier     |        1 |
| Bob     |        3 | Écran       |        2 |
| Charlie |        4 | Souris      |        3 |
+---------+----------+-------------+----------+
5 rows in set (0.10 sec)

mysql>
mysql> -- LEFT JOIN
mysql> SELECT c.nom, cmd.id AS num_commande
    -> FROM clients c
    -> LEFT JOIN commandes cmd ON c.id = cmd.client_id;
+---------+--------------+
| nom     | num_commande |
+---------+--------------+
| Alice   |            1 |
| Alice   |            2 |
| Bob     |            3 |
| Charlie |            4 |
| Dina    |         NULL |
+---------+--------------+
5 rows in set (0.07 sec)

mysql>
mysql> -- RIGHT JOIN
mysql> SELECT c.nom, cmd.id, cmd.date_commande
    -> FROM clients c
    -> RIGHT JOIN commandes cmd ON c.id = cmd.client_id;
```

Figure 5 – Résultats de l'INNER JOIN, du LEFT JOIN et début du RIGHT JOIN.

LEFT JOIN : tous les clients, avec ou sans commande

```
SELECT c.nom, cmd.id AS num_commande
FROM clients c
LEFT JOIN commandes cmd ON c.id = cmd.client_id;
```

Dina apparaît avec `NULL` : elle n'a passé aucune commande.

RIGHT JOIN

```
SELECT c.nom, cmd.id, cmd.date_commande
FROM clients c
RIGHT JOIN commandes cmd ON c.id = cmd.client_id;
```

Résultat : les 4 commandes avec leur client (toutes les commandes ont un client).

FULL JOIN (simulé avec UNION)

MySQL ne gère pas `FULL JOIN` : on combine un `LEFT JOIN` et un `RIGHT JOIN` avec `UNION`.

```
SELECT c.nom, cmd.id AS num_commande
FROM clients c LEFT JOIN commandes cmd ON c.id = cmd.client_id
UNION
SELECT c.nom, cmd.id AS num_commande
FROM clients c RIGHT JOIN commandes cmd ON c.id = cmd.client_id;
```

```
+---------+--------------+
| nom     | num_commande |
+---------+--------------+
| Alice   |            1 |
| Alice   |            2 |
| Bob     |            3 |
| Charlie |            4 |
| Dina    |         NULL |
+---------+--------------+
5 rows in set (0.09 sec)

mysql>
mysql> -- ====================================================
mysql> -- 3. Agrégations et calculs
mysql> -- ====================================================
mysql>
mysql> -- Montant total par commande
mysql> SELECT cmd.id AS commande, SUM(p.prix_unitaire * dc.quantite) AS total_commande
    -> FROM commandes cmd
    -> JOIN details_commandes dc ON cmd.id = dc.commande_id
    -> JOIN produits p ON dc.produit_id = p.id
    -> GROUP BY cmd.id;
+----------+----------------+
| commande | total_commande |
+----------+----------------+
|        1 |         990.00 |
|        2 |          45.00 |
|        3 |         500.00 |
|        4 |          60.00 |
+----------+----------------+
4 rows in set (0.18 sec)

mysql>
mysql> -- Montant total dépensé par client
mysql> SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total_client
    -> FROM clients c
    -> JOIN commandes cmd ON c.id = cmd.client_id
    -> JOIN details_commandes dc ON cmd.id = dc.commande_id
    -> JOIN produits p ON dc.produit_id = p.id
    -> GROUP BY c.nom
```

Figure 6 – Résultat du FULL JOIN simulé avec `UNION`.

6. Agrégations et calculs

Montant total par commande (`SUM` , `GROUP BY`)

```
SELECT cmd.id AS commande, SUM(p.prix_unitaire * dc.quantite) AS
total_commande
FROM commandes cmd
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY cmd.id;
```

commande	total_commande
1	990.00
2	45.00
3	500.00
4	60.00

```
    -> JOIN produits p ON dc.produit_id = p.id
    -> GROUP BY cmd.id;
+----------+----------------+
| commande | total_commande |
+----------+----------------+
|        1 |         990.00 |
|        2 |          45.00 |
|        3 |         500.00 |
|        4 |          60.00 |
+----------+----------------+
4 rows in set (0.18 sec)

mysql>
mysql> -- Montant total dépensé par client
mysql> SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total_client
    -> FROM clients c
    -> JOIN commandes cmd ON c.id = cmd.client_id
    -> JOIN details_commandes dc ON cmd.id = dc.commande_id
    -> JOIN produits p ON dc.produit_id = p.id
    -> GROUP BY c.nom
    -> HAVING total_client > 500;
+-------+--------------+
| nom   | total_client |
+-------+--------------+
| Alice |      1035.00 |
+-------+--------------+
1 row in set (0.09 sec)

mysql>
mysql> -- Montant moyen par produit
mysql> SELECT p.nom_produit, AVG(p.prix_unitaire) AS prix_moyen
    -> FROM produits p
    -> GROUP BY p.nom_produit;
+-------------+------------+
| nom_produit | prix_moyen |
+-------------+------------+
| Ordinateur  | 950.000000 |
| Souris      |  20.000000 |
| Clavier     |  45.000000 |
| Écran       | 250.000000 |
+-------------+------------+
```

Figure 7 – Agrégations : `SUM` avec `GROUP BY`, `HAVING` et `AVG`.

Montant dépensé par client (`HAVING`)

```
SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total_client
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY c.nom
HAVING total_client > 500;
```

Résultat : Alice (1035.00). `HAVING` filtre après l'agrégation, contrairement à `WHERE` .

Prix moyen par produit (`AVG`)

```
SELECT p.nom_produit, AVG(p.prix_unitaire) AS prix_moyen
FROM produits p
GROUP BY p.nom_produit;
```

7. Fonctions SQL

`IF()` : classer les clients

```
SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total,
       IF(SUM(p.prix_unitaire * dc.quantite) > 800, 'Premium', 'Standard') AS
categorie
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY c.nom;
```

nom	total	categorie
Alice	1035.00	Premium
Bob	500.00	Standard
Charlie	60.00	Standard

```
+-------------+------------+
4 rows in set (0.10 sec)

mysql>
mysql> -- ================================================
mysql> -- 4. Fonctions SQL
mysql> -- ================================================
mysql>
mysql> -- IF() pour classer les clients
mysql> SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total,
    ->        IF(SUM(p.prix_unitaire * dc.quantite) > 800, 'Premium', 'Standard') AS categorie
    -> FROM clients c
    -> JOIN commandes cmd ON c.id = cmd.client_id
    -> JOIN details_commandes dc ON cmd.id = dc.commande_id
    -> JOIN produits p ON dc.produit_id = p.id
    -> GROUP BY c.nom;
+---------+---------+-----------+
| nom     | total   | categorie |
+---------+---------+-----------+
| Alice   | 1035.00 | Premium   |
| Bob     |  500.00 | Standard  |
| Charlie |   60.00 | Standard  |
+---------+---------+-----------+
3 rows in set (0.08 sec)

mysql>
mysql> -- Fonctions de date
mysql> SELECT p.id AS paiement, cmd.id AS commande,
    ->        DATEDIFF(p.date_paiement, cmd.date_commande) AS delai_jours
    -> FROM paiements p
    -> JOIN commandes cmd ON p.commande_id = cmd.id;
+----------+----------+-------------+
| paiement | commande | delai_jours |
+----------+----------+-------------+
|        1 |        1 |           2 |
|        2 |        2 |           1 |
|        3 |        3 |           4 |
+----------+----------+-------------+
3 rows in set (0.09 sec)

mysql>
```

Figure 8 – Requêtes avec `IF()` et `DATEDIFF()`.

`DATEDIFF()` : délai de paiement

```
SELECT p.id AS paiement, cmd.id AS commande,
       DATEDIFF(p.date_paiement, cmd.date_commande) AS delai_jours
FROM paiements p
JOIN commandes cmd ON p.commande_id = cmd.id;
```

Délais obtenus : 2, 1 et 4 jours.

`LIMIT` : les 2 meilleurs clients

```
SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY c.nom
ORDER BY total DESC
LIMIT 2;
```

Résultat : Alice (1035.00) puis Bob (500.00).

```
mysql> -- LIMIT
mysql> SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total
    -> FROM clients c
    -> JOIN commandes cmd ON c.id = cmd.client_id
    -> JOIN details_commandes dc ON cmd.id = dc.commande_id
    -> JOIN produits p ON dc.produit_id = p.id
    -> GROUP BY c.nom
    -> ORDER BY total DESC
    -> LIMIT 2;
+-------+---------+
| nom   | total   |
+-------+---------+
| Alice | 1035.00 |
| Bob   |  500.00 |
+-------+---------+
2 rows in set (0.09 sec)

mysql>
mysql> -- ======================================================
mysql> -- 5. IN / NOT IN / EXISTS
mysql> -- ======================================================
mysql>
mysql> SELECT nom FROM clients WHERE id IN (
    ->   SELECT client_id FROM commandes WHERE id IN (
    ->     SELECT commande_id FROM paiements));
+-------+
| nom   |
+-------+
| Alice |
| Bob   |
+-------+
2 rows in set (0.11 sec)

mysql>
mysql> SELECT nom FROM clients WHERE id NOT IN (
    ->   SELECT client_id FROM commandes WHERE id IN (
    ->     SELECT commande_id FROM paiements));
```

Figure 9 – Requête avec `LIMIT 2` et début des sous-requêtes `IN`.

8. Opérateurs `IN`, `NOT IN` et `EXISTS`

```
-- Clients ayant au moins une commande payée
SELECT nom FROM clients WHERE id IN (
  SELECT client_id FROM commandes WHERE id IN (
    SELECT commande_id FROM paiements));

-- Clients sans commande payée
SELECT nom FROM clients WHERE id NOT IN (
  SELECT client_id FROM commandes WHERE id IN (
    SELECT commande_id FROM paiements));

-- Même résultat que IN, avec EXISTS
SELECT nom FROM clients c
WHERE EXISTS (
  SELECT 1 FROM commandes cmd
  JOIN paiements p ON cmd.id = p.commande_id
  WHERE cmd.client_id = c.id);
```

- `IN` et `EXISTS` : Alice et Bob
- `NOT IN` : Charlie et Dina (Charlie a une commande non payée, Dina n'a aucune commande)

```
+---------+
| nom     |
+---------+
| Charlie |
| Dina    |
+---------+
2 rows in set (0.08 sec)

mysql>
mysql> SELECT nom FROM clients c
    -> WHERE EXISTS (
    ->   SELECT 1 FROM commandes cmd
    ->   JOIN paiements p ON cmd.id = p.commande_id
    ->   WHERE cmd.client_id = c.id);
+-------+
| nom   |
+-------+
| Alice |
| Bob   |
+-------+
2 rows in set (0.09 sec)

mysql>
mysql> -- ====================================================
mysql> -- 6. Optimisation : Index et Vues
mysql> -- ====================================================
mysql>
mysql> CREATE INDEX idx_nom_produit ON produits(nom_produit);
Query OK, 0 rows affected (0.41 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_client_ville ON clients(ville);
Query OK, 0 rows affected (0.11 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
```

Figure 10 – Résultats de `NOT IN` et `EXISTS`, puis création des deux index.

9. Optimisation : index et vues

Index

```
CREATE INDEX idx_nom_produit ON produits(nom_produit);
CREATE INDEX idx_client_ville ON clients(ville);

EXPLAIN SELECT * FROM produits WHERE nom_produit = 'Souris';
```

Le plan d'exécution montre `type = ref`, `key = idx_nom_produit` et `rows = 1` : MySQL utilise l'index au lieu de parcourir toute la table (`type = ALL`).

```
mysql> EXPLAIN SELECT * FROM produits WHERE nom_produit = 'Souris';
+----+-------------+----------+------------+------+-----------------+-----------------+---------+-------+------+----------+-------+
| id | select_type | table    | partitions | type | possible_keys   | key             | key_len | ref   | rows | filtered | Extra |
+----+-------------+----------+------------+------+-----------------+-----------------+---------+-------+------+----------+-------+
|  1 | SIMPLE      | produits | NULL       | ref  | idx_nom_produit | idx_nom_produit | 203     | const |    1 |   100.00 | NULL  |
+----+-------------+----------+------------+------+-----------------+-----------------+---------+-------+------+----------+-------+
1 row in set, 1 warning (0.09 sec)

mysql>
mysql> CREATE VIEW vue_ventes AS
    -> SELECT c.nom, p.nom_produit, dc.quantite, (p.prix_unitaire * dc.quantite) AS montant, pa.mode_paiement
    -> FROM clients c
    -> JOIN commandes cmd ON c.id = cmd.client_id
    -> JOIN details_commandes dc ON cmd.id = dc.commande_id
    -> JOIN produits p ON dc.produit_id = p.id
    -> LEFT JOIN paiements pa ON cmd.id = pa.commande_id;
Query OK, 0 rows affected (0.18 sec)

mysql>
mysql> SELECT * FROM vue_ventes WHERE montant > 200;
+-------+-------------+----------+---------+---------------+
| nom   | nom_produit | quantite | montant | mode_paiement |
+-------+-------------+----------+---------+---------------+
| Alice | Ordinateur  |        1 |  950.00 | Carte         |
| Bob   | Écran       |        2 |  500.00 | Virement      |
+-------+-------------+----------+---------+---------------+
2 rows in set (0.10 sec)

mysql>
mysql> -- ======================================================
mysql> -- 7. DCL – Gestion des utilisateurs
mysql> -- ======================================================
mysql>
mysql> CREATE USER 'vendeur'@'localhost' IDENTIFIED BY '1234';
Query OK, 0 rows affected (0.18 sec)

mysql> CREATE USER 'manager'@'localhost' IDENTIFIED BY 'admin';
Query OK, 0 rows affected (0.10 sec)

mysql>
mysql> GRANT SELECT, INSERT, UPDATE ON atelier_commerce.* TO 'vendeur'@'localhost';
```

Figure 11 – `EXPLAIN` (index utilisé), création de la vue `vue_ventes` et de ses résultats, création des utilisateurs.

Vue

```
CREATE VIEW vue_ventes AS
SELECT c.nom, p.nom_produit, dc.quantite,
       (p.prix_unitaire * dc.quantite) AS montant, pa.mode_paiement
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
LEFT JOIN paiements pa ON cmd.id = pa.commande_id;

SELECT * FROM vue_ventes WHERE montant > 200;
```

nom	nom_produit	quantite	montant	mode_paiement
Alice	Ordinateur	1	950.00	Carte
Bob	Écran	2	500.00	Virement

La vue simplifie une requête à 5 tables en un simple `SELECT`.

La vue est créée puis interrogée dans la Figure 11.

10. Sécurité : gestion des droits (DCL)

```
CREATE USER 'vendeur'@'localhost' IDENTIFIED BY '1234';
CREATE USER 'manager'@'localhost' IDENTIFIED BY 'admin';

GRANT SELECT, INSERT, UPDATE ON atelier_commerce.* TO 'vendeur'@'localhost';
GRANT ALL PRIVILEGES ON atelier_commerce.* TO 'manager'@'localhost';
FLUSH PRIVILEGES;

SHOW GRANTS FOR 'manager'@'localhost';
REVOKE UPDATE ON atelier_commerce.* FROM 'vendeur'@'localhost';
```

- `vendeur` : lecture, ajout et modification uniquement (principe du moindre privilège), puis retrait de `UPDATE`
- `manager` : tous les droits sur `atelier_commerce`
- `SHOW GRANTS` confirme `ALL PRIVILEGES` sur `atelier_commerce` pour `manager`

Les mots de passe `1234` et `admin` ne servent que pour l'exercice : en production, on utilise des mots de passe forts.

```
mysql>
mysql> GRANT SELECT, INSERT, UPDATE ON atelier_commerce.* TO 'vendeur'@'localhost';
Query OK, 0 rows affected, 1 warning (0.10 sec)

mysql> GRANT ALL PRIVILEGES ON atelier_commerce.* TO 'manager'@'localhost';
Query OK, 0 rows affected, 1 warning (0.10 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.10 sec)

mysql>
mysql> SHOW GRANTS FOR 'manager'@'localhost';
+-----------------------------------------------------------------------+
| Grants for manager@localhost                                          |
+-----------------------------------------------------------------------+
| GRANT USAGE ON *.* TO "manager"@"localhost"                           |
| GRANT ALL PRIVILEGES ON "atelier_commerce".* TO "manager"@"localhost" |
+-----------------------------------------------------------------------+
2 rows in set (0.10 sec)

mysql>
mysql> REVOKE UPDATE ON atelier_commerce.* FROM 'vendeur'@'localhost';
```

Figure 12 – `GRANT`, `SHOW GRANTS` pour `manager` et `REVOKE` pour `vendeur`.

11. Difficultés rencontrées

- **Erreur 1045 (Access denied, using password: NO)** : le mot de passe n'était pas transmis. Solution : ajouter l'option `-p` à la commande.
- **Conflit de services MySQL en local** : plusieurs serveurs MySQL écoutaient sur le port 3306. Choisir un serveur distant (Aiven) a contourné le problème et correspond à la consigne de l'atelier.

12. Conclusion et compétences acquises

Cet atelier m'a permis de concevoir une base relationnelle complète et de l'exploiter sur un serveur MySQL cloud.

- Modélisation et création de schéma (clés primaires et étrangères)
- Jointures `INNER`, `LEFT`, `RIGHT` et `FULL` simulée
- Agrégations (`SUM`, `AVG`, `GROUP BY`, `HAVING`)
- Fonctions `IF()`, `DATEDIFF()` et `LIMIT`
- Sous-requêtes (`IN`, `NOT IN`, `EXISTS`)

- Optimisation (index, `EXPLAIN`) et vues
- Gestion des utilisateurs et des droits (`CREATE USER`, `GRANT`, `REVOKE`)
- Connexion sécurisée (SSL) à un service MySQL hébergé sur Aiven